STARVNT

CUSTOMER E2E PRODUCT BLUEPRINT v1.0

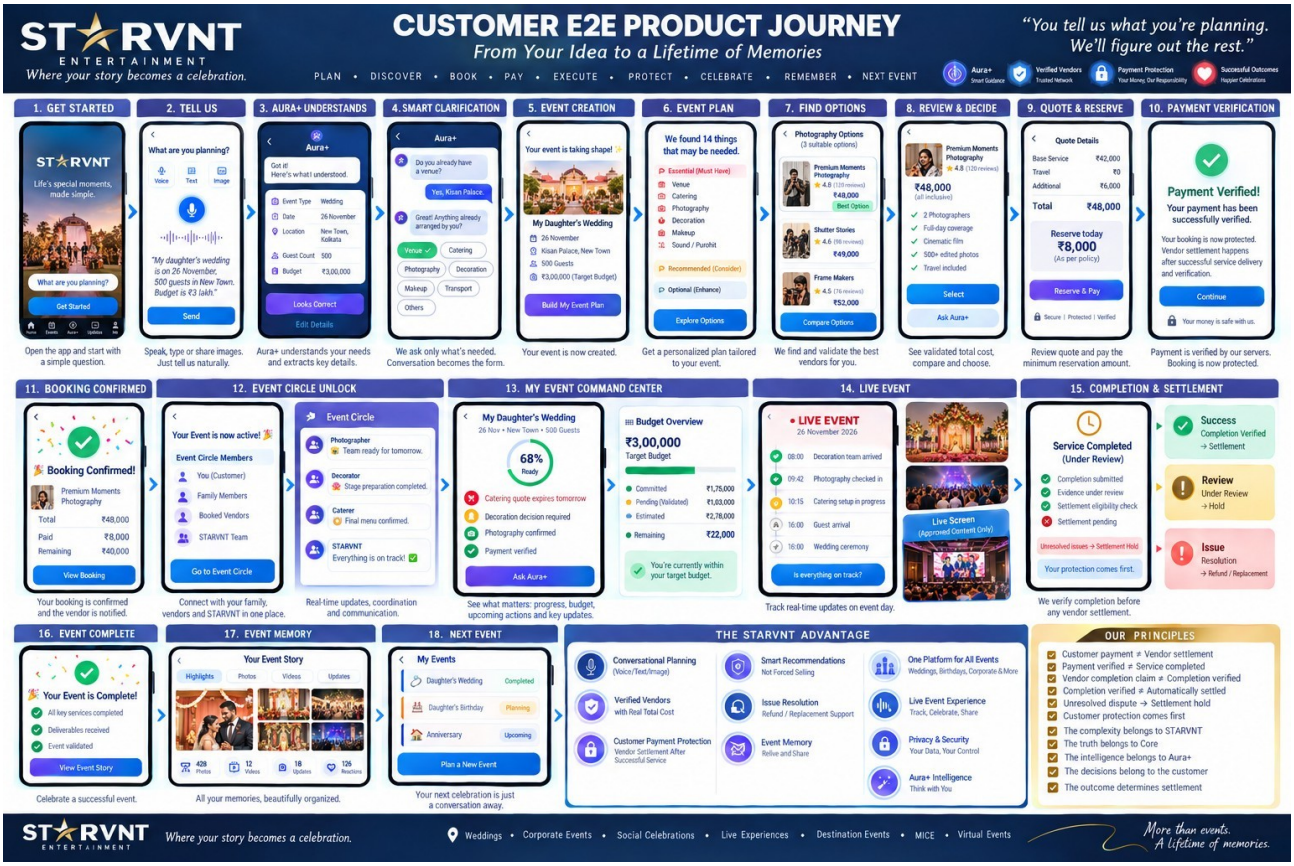
Personal Event Assistant -> Commerce -> Protected Booking -> Execution -> Outcome -> Settlement -> Event Memory

PRODUCT PROMISE

You tell us what you're planning. We'll figure out the rest.

NORTH STAR

The complexity belongs to STARVNT. The decision belongs to the customer. The truth belongs to Core. The outcome determines settlement.



Reference: supplied Customer E2E journey concept.

Document Purpose

This document is an implementation-facing product contract. It is written so a product engineer, frontend engineer, backend engineer, AI engineer, QA engineer, DevOps engineer, or technical lead can understand what STARVNT must do, what it must never do, which system is authoritative, and how the customer journey maps to event state, commerce state, payment state, execution state, protection, settlement, and memory.

The supplied UI concepts are treated as experience references, not as the source of business truth. The UI must render authoritative state from Core and approved APIs. It must not create business truth by itself.

Table of Contents

1. 1. Product Contract and Authority Model
2. 2. Customer E2E Journey
3. 3. Event and Requirement Model
4. 4. Discovery, Validation, Quotes and Decisions
5. 5. Payment, Booking and Customer Protection
6. 6. Vendor Execution, Completion and Settlement
7. 7. Event Circle, Live Event and Privacy
8. 8. Aura+ Intelligence Contract
9. 9. Customer App Information Architecture
10. 10. Core Data Model and State Machines
11. 11. API / Backend Behaviour Contract
12. 12. Security, Authorization and Audit
13. 13. Reliability, Idempotency and Failure Handling
14. 14. UI Implementation Rules
15. 15. E2E Testing and Acceptance Criteria
16. 16. Non-Goals and Explicit Anti-Patterns
17. 17. Developer Build Order
18. 18. Reference Screens and Product Language

1. PRODUCT CONTRACT AND AUTHORITY MODEL

1.1 Four immutable authority principles

Layer	Authority	May do	Must not do
Customer	Decision authority	Choose, approve, edit, reject, report issues, control sharing	Be silently overridden by AI or system recommendations
Aura+	Intelligence authority	Understand, clarify, explain, summarize, recommend, detect missing context	Declare payment success, verify completion, authorize settlement, invent facts, silently change bookings
Core	System-of-record authority	Persist and enforce authoritative event, requirement, quote, booking, payment and execution truth	Let frontend state become authoritative
Policy / Payment / Settlement infrastructure	Rule / money-movement authority	Apply commercial rules and process real money movement as configured	Be replaced by chat logic or UI shortcuts

LOCK THIS RULE

Aura+ can understand and recommend. Core must determine what is officially true.

1.2 Customer-facing journey vs internal complexity

Customer: TELL -> UNDERSTAND -> PLAN -> DISCOVER -> COMPARE -> DECIDE -> RESERVE -> PAY -> BOOK -> CONNECT -> COORDINATE -> EXPERIENCE -> COMPLETE -> REMEMBER

Internal: Intent -> Event -> Requirement -> Service Location -> Eligibility -> Commercial -> Quote -> Decision -> Reservation -> Payment -> Verification -> Booking -> Protection -> Execution -> Evidence -> Validation -> Settlement/Recovery -> Memory

1.3 Golden product rules

- Customer payment is not vendor settlement.
- Payment confirmed is not service completed.
- Vendor completion claim is not completion verified.
- Completion verified is not automatically settlement executed.
- An unresolved applicable service issue keeps the settlement decision under review according to policy.
- Customer claims trigger review; they do not automatically establish vendor fault.
- Vendor claims trigger validation; they do not automatically establish success.
- The customer decides; STARVNT recommends and coordinates.
- The UI is not a security boundary. The backend must enforce access.
- One event can contain many requirements, quotes, bookings and service-specific execution states.

2. CUSTOMER E2E JOURNEY

#	Stage	Implementation intent
01	Welcome	Life's special moments, made simple. Primary CTA: Get Started. Do not force a complex dashboard before the customer states intent.
02	What are you planning?	Support voice, text, media and quick prompts. Conversation becomes the form.
03	Aura+ understanding	Extract event type, date, guest count, destination/venue, budget, already-arranged services and stated needs.
04	Smart clarification	Ask only what is required to reduce meaningful ambiguity. Do not interrogate the customer with a long form.
05	Event creation	Create an Event only when sufficient context exists. Show what was understood before committing important assumptions.
06	Event plan	Generate essential, recommended and optional requirements. Recommendations are not purchases.
07	Find options	Resolve each requirement into a service location, eligibility, availability and validated total cost.
08	Review and decide	Show comparable options, transparent inclusions and explain why an option is recommended. The customer decides.
09	Quote and reserve	Create a quote and calculate a policy-driven minimum reservation. Never hardcode a universal deposit.
10	Payment verification	Initiate payment, verify server-side, reconcile webhook/callbacks and only then allow payment-confirmed state.
11	Booking confirmed	Create a confirmed booking from Core. Payment confirmation alone is not enough if booking rules are not satisfied.
12	Event Circle	Unlock controlled collaboration after verified booking, with permissioned members and need-to-know access.
13	Command Center	Show what matters now: readiness, budget, upcoming actions, vendor status, payment status and risks.
14	Live Event	Simplify the customer UI to operational

#	Stage	Implementation intent
		event-day status and on-track answers based only on known Core data.
15	Completion and settlement	Collect evidence, validate completion, apply policy, place holds when required, and settle only when eligible.
16	Event Memory	Turn the completed event into an organized story, photos, videos, updates, approved content and future-event context.

CUSTOMER ABSTRACTION

The customer should never feel they are operating a Requirement module, Quote service, Settlement engine, or State machine. They are planning an event. STARVNT manages the machinery.

3. EVENT AND REQUIREMENT MODEL

3.1 Event is the long-lived customer context

CUSTOMER
-> EVENT
-> event type
-> date(s)
-> destination / venue context
-> guest context
-> budget context
-> requirements
-> decisions
-> bookings
-> execution
-> completion
-> memory

3.2 Requirement is the central operational object

Field	Meaning
Requirement ID	Stable ID for one requested service need inside an Event
Event ID	Parent Event
Service category	Photography, decoration, catering, venue, transport, etc.
Service definition	The concrete service/package being sourced
Service location	Where the service must actually happen
Date / duration	Service-specific timing
Quantity / capacity	Photographers, guests, vehicles, rooms, etc.

Field	Meaning
Coverage / constraints	Style, deliverables, access, timing, inclusions, exclusions
Budget context	Target / ceiling / flexibility; not always a hard limit
Status	Authoritative lifecycle state in Core
Decision history	Customer selections/rejections and why
Risk flags	Availability, price, timing, vendor, or execution risks

3.3 Requirement lifecycle

IDENTIFIED -> CLARIFICATION_REQUIRED -> READY -> DISCOVERING -> OPTIONS_FOUND -> QUOTE_REQUESTED -> QUOTE_RECEIVED -> CUSTOMER_REVIEW -> SELECTED -> BOOKED -> SCHEDULED -> EXECUTING -> COMPLETION_SUBMITTED -> COMPLETION_VERIFIED -> CLOSED

3.4 Service location principle

DO NOT MATCH ON CUSTOMER LOCATION ALONE

Event Location != Service Location. Photography may be at the venue; makeup may be in a bridal room; transport may be airport -> venue. Vendor matching must use the actual service location and operational context.

4. DISCOVERY, VALIDATION, QUOTES AND DECISIONS

4.1 Vendor discovery pipeline

REQUIREMENT
-> SERVICE LOCATION
-> SERVICE
-> COVERAGE
-> AVAILABILITY
-> CAPACITY
-> ELIGIBILITY
-> TRAVEL
-> ADDITIONAL COSTS
-> TOTAL VALIDATED COST
-> SUITABILITY
-> CUSTOMER-FACING OPTIONS

4.2 Lowest validated total cost

STARVNT must not optimize against a bare advertised price. The comparison basis is the validated total cost among vendors that genuinely satisfy the requirement, subject to availability, coverage and operational eligibility.

Option	Base	Travel	Additional	Validated total	Eligibility
Vendor A	44,000	3,000	2,000	49,000	Meets

Option	Base	Travel	Additional	Validated total	Eligibility
					requirement
Vendor B	48,000	Included	Included	48,000	Meets requirement

4.3 Recommendation contract

Aura+ may explain why an option is suitable. The commercial engine/Core must provide the facts behind the explanation: validated cost, included coverage, availability, travel treatment, capacity, and other rule-based inputs.

RECOMMENDATION RULE

A recommended option is not a forced option. The UI must always preserve customer choice and expose comparison or details where applicable.

4.4 Protected discovery

- Before booking, protect customer PII and unnecessary vendor information.
- Use an internal opportunity/quote identifier for marketplace coordination.
- After booking, reveal only operational details required for delivery.
- Keep communication attached to Event + Requirement + Vendor + Quote/Booking context.

4.5 Quote lifecycle

```
DRAFT -> SUBMITTED -> CUSTOMER_REVIEW -> APPROVED
                          |
                          +-> REJECTED
                          +-> EXPIRED
```

4.6 Reservation

Reservation amount is policy-driven. The calculation can depend on service type, vendor terms, event timing, risk, booking value, cancellation rules, and configured commercial policy. There is no universal hardcoded reservation amount.

5. PAYMENT, BOOKING AND CUSTOMER PROTECTION

5.1 Payment architecture rule

```
PAYMENT_VERIFICATION != FUNDS_CUSTODY != SETTLEMENT_ELIGIBILITY != SETTLEMENT_EXECUTION
```

The software must model these as distinct concepts. Actual custody and settlement mechanisms depend on the selected payment/marketplace infrastructure and applicable legal/compliance structure. Do not label the system as “escrow” unless the underlying legal and technical arrangement genuinely supports that construct.

5.2 Payment state machine

```
PAYMENT_INITIATED -> PAYMENT_PROCESSING -> PAYMENT_CONFIRMED
                          |
```

+-> PAYMENT_FAILED
+-> PAYMENT_REVERSED
+-> PAYMENT_REFUNDED

5.3 Verification

19. Client requests reservation and the server creates a payment intent / order using the configured payment infrastructure.
20. Client is redirected or presented with the gateway UI.
21. Frontend success callback is informational, not authoritative.
22. Server verifies the gateway result and receives/validates webhook or provider notification.
23. Server reconciles amount, currency, booking reference, customer reference and idempotency key.
24. Only then does Core mark the payment as PAYMENT_CONFIRMED.
25. Booking confirmation is created only after booking preconditions are satisfied.

MUST NOT

Do not set paymentConfirmed=true from client-only success callbacks. Do not release vendor settlement because a customer payment is successful.

5.4 Booking state machine

RESERVATION_REQUESTED -> PAYMENT_CONFIRMED -> BOOKING_CONFIRMED -> SERVICE_SCHEDULED -> SERVICE_STARTED -> SERVICE_COMPLETED

5.5 Customer Protection case

State	Meaning
NONE	No protection issue open
AT_RISK	Booking/execution condition indicates risk
PROTECTED	Booking is under applicable protection framework
ISSUE_REPORTED	Customer/vendor has raised a service issue
UNDER_REVIEW	Evidence and policy are being evaluated
RESOLUTION_PROPOSED	A resolution is prepared according to rules
RESOLVED	Case is closed with an auditable outcome

5.6 Customer issue flow

CUSTOMER ISSUE
-> EVENT
-> REQUIREMENT
-> BOOKING
-> VENDOR
-> PAYMENT
-> COMPLETION
-> EVIDENCE
-> POLICY

-> REVIEW
-> RESOLUTION

- A claim starts review; it does not prove fault.
- A vendor completion claim starts validation; it does not prove completion.
- Unresolved applicable disputes can place settlement on hold according to policy.
- Possible outcomes may include service acceptance, settlement, partial settlement, refund, partial refund, replacement, or another approved remedy.

6. VENDOR EXECUTION, COMPLETION AND SETTLEMENT

6.1 Vendor execution lifecycle

BOOKING_CONFIRMED -> PAYMENT_VERIFIED -> SERVICE_SCHEDULED -> CHECK_IN -> SERVICE_STARTED -> MILESTONE -> SERVICE_COMPLETED -> COMPLETION_SUBMITTED

6.2 Completion is not settlement

State	What it means	Who can cause it	What it does NOT mean
PAYMENT_CONFIRMED	Customer payment verified	Payment/Core	Vendor is paid
SERVICE_COMPLETED	Operational service ended	Vendor submits / system observes	Completion is verified
COMPLETION_VERIFIED	Core accepts required completion conditions	Core/authorized operations	Money is automatically settled
SETTLEMENT_ELIGIBLE	Policy says settlement conditions are satisfied	Core + Policy	Money has moved
VENDOR_SETTLED	Actual settlement executed	Settlement infrastructure	No future dispute can exist

6.3 Evidence

- Check-in / check-out signals where available
- Service photos or deliverables
- Milestone completion
- Documents or delivery proof
- Customer acknowledgement where the service requires it
- Category-specific evidence configured by policy

6.4 Service-specific settlement

Not every vendor should be settled on the same trigger. Photography may require coverage plus specific deliverable conditions. Catering may be complete after event-day service. Decoration may require setup, event service and dismantling. Settlement criteria are therefore service-specific, while customer protection remains universal.

6.5 Settlement state machine

NOT_ELIGIBLE -> ELIGIBILITY_CHECK -> ELIGIBLE -> SETTLEMENT_PENDING -> SETTLEMENT_PROCESSING -> VENDOR_SETTLED
|
+-> DISPUTED -> HOLD -> REVIEW -> RESOLUTION

AUTHORITY RULE

Vendor cannot self-authorize settlement. Customer cannot authorize vendor settlement. Aura+ cannot authorize settlement. Frontend cannot set settlementEligible=true. Core + policy determine eligibility; settlement infrastructure moves money.

7. EVENT CIRCLE, LIVE EVENT AND PRIVACY

7.1 Event Circle

After an eligible verified booking, create a private collaboration layer around the Event. Members can include the customer, authorized family/participants, booked vendors and STARVNT personnel. This is not a social network; it is a permissioned event coordination space.

7.2 Access levels

Stage	Default access
Pre-booking	Customer + STARVNT; vendor identity/business info only as appropriate
Post-booking	Customer + booked vendor with need-to-know operational access
Event Circle	Customer-selected family/participants + booked vendors + authorized STARVNT roles
Public	Only when customer explicitly publishes

7.3 Progressive location reveal

BEFORE BOOKING: New Town, Kolkata
PLANNING: Kisan Palace, New Town
AFTER BOOKING: Exact operational location
NEAR EVENT: Arrival point + access instructions + contact person

7.4 Live Event mode

On event day, the customer experience shifts from planning to tracking. The customer needs concise operational status rather than the full planning interface.

08:00 Decoration team arrived [GREEN]
09:42 Photography checked in [GREEN]
10:15 Catering setup in progress [AMBER]
16:00 Guest arrival [PLANNED]
18:00 Wedding ceremony [PLANNED]

7.5 “Is everything on track?”

Aura+ may answer only from current Core state and approved operational data. If a fact is unavailable, say so explicitly. Never infer a vendor check-in, completion, or timing that the system does not know.

7.6 Live screen safety

GUEST UPLOAD -> PENDING -> PRIVACY CHECK -> SAFETY CHECK -> EVENT PERMISSION -> APPROVAL/MODERATION -> PUBLISHED -> LIVE SCREEN ELIGIBLE

MUST NOT

Never route raw guest uploads directly to an LED/event screen.

8. AURA+ INTELLIGENCE CONTRACT

8.1 Aura+ can

- Understand natural language, voice and media context
- Classify known / confirmed / inferred / missing / ambiguous / recommended data
- Ask smart clarification questions
- Summarize event context
- Explain validated options and trade-offs
- Recommend suitable next actions
- Detect missing requirements from actual Event state
- Explain current execution status
- Help recover from operational issues

8.2 Aura+ must not

- Declare authoritative payment success
- Verify a gateway transaction independently of Core/payment infrastructure
- Create unauthorized bookings
- Change customer decisions silently
- Verify service completion on intuition alone
- Authorize settlement
- Decide refunds independently of policy and Core
- Invent vendor availability, pricing, check-ins, evidence or completion status
- Override security rules
- Expose protected data because a customer asked for it in chat

8.3 Explanation pattern

CORE / COMMERCIAL FACTS -> AURA+ EXPLANATION -> CUSTOMER DECISION

8.4 Context model

Context class	Meaning	Persistence / rule
Known	Customer explicitly stated or data already in Core	May be stored as observed fact
Confirmed	Customer explicitly approved/verified	High-authority customer input
Inferred	AI interpretation not yet confirmed	Must be clearly marked and never treated as booking truth

Context class	Meaning	Persistence / rule
Missing	Required information not available	May trigger clarification
Ambiguous	Conflicting or unclear information	Must trigger clarification or safe fallback
Recommended	Useful but optional suggestion	Never auto-book or auto-add without decision

9. CUSTOMER APP INFORMATION ARCHITECTURE

9.1 Primary navigation

Primary item	Purpose
Home	What matters now for the customer across active events
Events	All events and each event command center
Aura+	Think with me - natural language interaction
Updates	Meaningful changes and actions needing attention
Me	Identity, privacy, security, settings and support

DO NOT MAKE THESE PRIMARY NAV ITEMS

Requirements, Vendors, Quotes, Payments, Budget, Timeline, Documents. Keep them inside Event Context unless a global view is genuinely required for a specific cross-event task.

9.2 Event Command Center

- Event name/date/location/guest summary
- Readiness percentage or status
- Requirement cards
- Upcoming actions
- Vendor status
- Payment status
- Budget overview
- Event timeline
- Contextual messages
- Ask Aura+
- Event Circle
- Documents and approved memory

9.3 Attention Engine

The customer should see only meaningful updates: expiring quotes, decisions required, verified payments, vendor readiness, schedule changes, issues, and event-day check-ins. Avoid notification spam.

10. CORE DATA MODEL AND STATE MACHINES

10.1 Minimum domain entities

Entity	Purpose
Customer / Identity	Authenticated customer and privacy preferences
Organization / Tenant	Business or event-operator boundary if multi-tenant
Event	Long-lived planning and execution context
Requirement	One service need in an Event
Service	Sellable definition/package/category
Vendor	Provider business identity and operational capability
VendorService	Vendor-specific service offering
Quote	Commercial offer for a Requirement
Booking	Customer decision converted into an execution commitment
Payment	Payment lifecycle separate from Booking
Execution	Operational delivery lifecycle
Completion	Completion claim + evidence + verification
ProtectionCase	Customer protection / issue / dispute lifecycle
Settlement	Eligibility and money movement status
EventCircle	Permissioned event collaboration
Message	Contextual communication
Notification	Meaningful event-driven attention item
AuditEntry	Immutable or append-only record of critical state changes
MemoryAsset	Photos, videos, approved updates and event story data

10.2 State separation

PAYMENT STATE
INITIATED / PROCESSING / CONFIRMED / FAILED / REVERSED / REFUNDED

BOOKING STATE
RESERVATION_REQUESTED / CONFIRMED / SCHEDULED / STARTED / COMPLETED

COMPLETION STATE
SUBMITTED / UNDER_REVIEW / VERIFIED / REJECTED

SETTLEMENT STATE
NOT_ELIGIBLE / ELIGIBILITY_CHECK / ELIGIBLE / PENDING / PROCESSING / SETTLED / HOLD / RESOLVED

10.3 Event state

DRAFT -> PLANNING -> BOOKING_IN_PROGRESS -> READY_FOR_EVENT -> LIVE -> COMPLETION_REVIEW -> COMPLETED -> MEMORY_AVAILABLE -> ARCHIVED (optional)

10.4 Event completion rule

Event completion is not “date has passed.” The Event must satisfy configured completion conditions for its relevant requirements/services, and unresolved critical issues must be handled according to policy.

11. API / BACKEND BEHAVIOUR CONTRACT

11.1 API principles

- All mutations must be server-authoritative.
- Return explicit current state, allowed next actions and version/timestamp where useful.
- Use idempotency keys for payments, bookings, event creation and other retryable critical commands.
- Validate tenant, customer, event and role access on every protected mutation/query.
- Never trust client-provided financial state, settlement state or security claims.
- Prefer command-style endpoints for critical state transitions where semantics matter.

11.2 Example command APIs

Command	Expected behaviour
POST /events	Create one Event if validated and idempotency-safe
POST /events/{id}/requirements	Create or update Requirement with authorization and validation
POST /requirements/{id}/discover	Start vendor discovery using current requirement state
POST /quotes/{id}/approve	Record customer decision; validate quote still valid
POST /bookings/{id}/reserve	Create reservation/payment intent according to policy
POST /payments/{id}/verify	Server-only verification/reconciliation path
POST /webhooks/payment	Idempotent provider callback handling
POST /bookings/{id}/report-issue	Create ProtectionCase and evaluate settlement hold
POST /executions/{id}/completion	Vendor completion submission + evidence
POST /completions/{id}/verify	Core-authorized completion decision
POST /settlements/{id}/execute	Settlement infrastructure command; never from client UI
POST /event-circles/{id}/members	Permissioned membership changes
POST /memory/{id}/publish	Publish approved memory assets

11.3 Response contract

```
{
 "data": {...},
 "state": "AUTHORITATIVE_STATE",
 "allowedActions": ["..."],
```

```
  "version": 123,
  "updatedAt": "...",
  "requestId": "..."
}
```


11.4 Optimistic UI

Optimistic UI may temporarily show intent, but it must reconcile to authoritative server state. Never label payment, booking confirmation, completion verification or settlement as final solely from optimistic state.

12. SECURITY, AUTHORIZATION AND AUDIT

12.1 Authorization model

- Customer can access only their own authorized Events and Event Circle membership.
- Vendor can access only authorized opportunities/bookings and vendor-owned resources.
- STARVNT internal roles should be scoped by organization, Event, or operational permission.
- Security must be enforced server-side; hiding a UI button is not authorization.
- Use least-privilege data projection: return only fields needed by the current actor and stage.

12.2 Progressive data access

Data	Pre-booking	Post-booking	Event Circle / Ops
Customer name/contact	Masked / minimized	As required for delivery	As permission allows
Exact service location	Limited	Operationally available	Operationally available
Vendor private financial info	Never	Never	Never unless authorized
Booking/commercial data	Quote-level	Booking-level	Relevant operational detail
Evidence	No	As relevant	Authorized participants only

12.3 Audit requirements

Every critical state transition should record who/what/when/from/to/reason/source/authority/reference/idempotency key. Audit is required for payment, booking, quote approval, completion verification, dispute/hold, refund, settlement, privacy changes and access/security events.

AUDIT ENTRY
WHO | WHAT | WHEN | FROM | TO | WHY | SOURCE | AUTHORITY | REFERENCE | IDEMPOTENCY KEY

13. RELIABILITY, IDEMPOTENCY AND FAILURE HANDLING

13.1 Idempotency

ENGINEERING PRINCIPLE

One valid business request should produce one valid business outcome, even when the request is retried or delivered more than once.

Failure / risk	Expected outcome
Payment gateway callback duplicated	One payment confirmation outcome
Double tap booking	One booking or safe conflict response
Create event request repeated	One Event per idempotency intent
Event Circle creation retried	One Event Circle
Notification worker retried	No duplicate customer-facing notification when dedupe is required
Automation retried	One intended business transition
Vendor cancellation	Protection + replacement/recovery path
Customer issue	Protection case + settlement review/hold according to policy
Service incomplete	Evidence review + resolution path
Cross-tenant access	Denied
Unauthorized vendor access	Denied
Unapproved live content	Cannot reach Live Screen

13.2 Recovery

- Use durable queues for asynchronous work where appropriate.
- Use retry policies with bounded attempts and dead-letter handling.
- Store provider reference IDs for reconciliation.
- Reconcile financial state from the payment provider rather than trusting a single callback.
- Make background jobs safe to rerun.
- Keep state transitions explicit and validate allowed source states.

13.3 Invalid state protection

If a command is sent from an invalid state, reject it with a stable domain error and current authoritative state. Never “force through” a transition simply because the frontend button exists.

14. UI IMPLEMENTATION RULES

14.1 General

- Render from API state; do not derive authoritative state only in React state.
- Keep loading, empty, error, stale, pending and success states explicit.
- Every critical mutation must have an in-progress state and server reconciliation.
- Show customer-friendly language, but keep machine-readable error/state codes available for telemetry and QA.
- Avoid exposing internal IDs or technical terminology unless useful for support/audit views.

14.2 Payment UI

- “Payment successful” means provider/server verification succeeded, not merely browser callback.

- After payment, explicitly show booking status separately.
- Do not show vendor settlement status to customers unless product policy requires a customer-facing summary.
- Do not promise refund completion until the underlying refund state is confirmed.

14.3 Aura+ UI

- Clearly distinguish “I understood” from “I confirmed.”
- When using inferred information, ask for confirmation before it affects a commercial action.
- When operational data is missing, say so.
- When recommending, show the reason and trade-offs from validated data.
- Never fabricate current status.

14.4 Event Day UI

- Prioritize current time, next milestone, on-track risk and team arrivals.
- Collapse low-priority planning detail.
- Provide clear issue/report action.
- Do not overwhelm customers with internal execution logs.

15. E2E TESTING AND ACCEPTANCE CRITERIA

15.1 Golden happy path

Fresh customer
-> “My daughter's wedding is on 26 November in New Town for 500 guests with a 3 lakh budget.”
-> Aura+ understands
-> clarification
-> Event created
-> Event Plan
-> Photography Requirement
-> Service Location
-> Vendor discovery
-> validated options
-> compare
-> customer selects
-> quote
-> reservation
-> payment
-> server verification
-> booking confirmed
-> Event Circle
-> vendor execution
-> evidence
-> completion verification
-> settlement eligibility
-> settlement
-> Event Completed
-> Event Memory

15.2 E2E-02 Vendor cancellation

Booked -> Vendor Cancelled -> Booking At Risk -> Customer Protection -> Alternative Discovery -> Validated Options -> Customer Decision -> Replacement / Applicable Resolution

15.3 E2E-03 Service dispute

Paid -> Booked -> Executed -> Customer Issue -> Settlement Hold -> Evidence -> Review -> Resolution -> Settlement/Refund/Replacement/Other

15.4 E2E-04 Payment failure

Payment Initiated -> Failed -> Retry -> Success -> Verify -> One Booking

15.5 E2E-05 Duplicate safety

Duplicate webhook + duplicate booking + duplicate automation + duplicate Event Circle creation -> exactly one valid business outcome

15.6 E2E-06 Security

Customer A -> Event B -> DENIED
Vendor A -> Vendor B booking -> DENIED
Frontend attempts settlement authorization -> REJECTED

15.7 Acceptance criteria checklist

Area	Acceptance criteria
Customer intent	Voice/text/media input produces a structured context with confidence/confirmation distinctions
Clarification	System asks the minimum useful questions and remembers answers in Event context
Event	Every post-creation customer action is tied to an Event
Requirement	Each service need has its own lifecycle
Discovery	Options use service location, availability, eligibility and validated total cost
Decision	Customer approval/rejection is explicit and auditable
Payment	Client callbacks are not authoritative; server verification + webhook reconciliation are required
Booking	Booking is separate from payment state
Protection	Customer issue creates a case and can place applicable settlement on hold
Execution	Vendor can submit completion/evidence but cannot settle themselves
Completion	Core verifies according to service-specific conditions
Settlement	Eligibility is distinct from actual settlement
Security	All protected routes enforce actor/event/tenant authorization
Reliability	Critical commands are idempotent and safe to retry
Aura+	Never fabricates truth or performs unauthorized financial

Area	Acceptance criteria
	actions
Memory	Completed events can be retrieved without losing their operational history

16. NON-GOALS AND EXPLICIT ANTI-PATTERNS

16.1 MUST NOT build

- No client-side source of truth for payment or settlement.
- No hardcoded one-size-fits-all reservation amount.
- No “payment success = vendor paid” shortcut.
- No “vendor clicked complete = settlement” shortcut.
- No “date passed = event completed” shortcut.
- No automatic booking of AI recommendations without customer decision unless an explicit user-controlled autopilot feature exists and is separately consented.
- No raw guest upload directly to Live Screen.
- No exposure of private customer data to vendors before it is operationally necessary.
- No generic inbox detached from Event/Requirement/Booking context.
- No separate hardcoded backend for Wedding vs Birthday vs Corporate; use universal Event/Requirement architecture.
- No silent mutation of customer choices by Aura+.
- No security model based only on hidden UI elements.

16.2 Anti-pattern examples

Bad implementation	Required replacement
setPaymentConfirmed(true) in browser on success callback	Server verify + webhook + reconciliation
setSettlementEligible(true) after payment	Evaluate service completion + policy in Core
vendor completion button immediately triggers payout	Completion submission -> evidence -> verification -> eligibility -> settlement
AI says “photographer checked in” because timeline says 9:42	Read authoritative check-in state or say “not confirmed”
recommendation stored as booking	Store recommendation separately; require customer decision
all event data visible to everyone in Event Circle	Permissioned projections and role-specific access
one giant Event status field controls every sub-process	Use separate state machines for Event, Requirement, Payment, Booking, Completion and Settlement

17. DEVELOPER BUILD ORDER

Phase 1 - Foundation

- Identity / Customer / Tenant authorization
- Event entity + event lifecycle
- Requirement entity + lifecycle
- Audit framework
- Idempotency infrastructure
- Shared API state model

Phase 2 - Planning + Discovery

- Aura+ intent/context extraction
- Clarification engine
- Event Plan engine
- VendorService/availability model
- Service Location model
- Validated total cost calculation
- Option comparison

Phase 3 - Commerce

- Quote lifecycle
- Customer decision recording
- Reservation policy
- Payment intent/order creation
- Server verification
- Webhook reconciliation
- Booking confirmation

Phase 4 - Protection + Execution

- Execution lifecycle
- Check-in/milestones
- Completion evidence
- Protection case
- Issue reporting
- Settlement hold
- Policy-driven resolution

Phase 5 - Event Circle + Live

- Event Circle membership
- Contextual messaging
- Progressive access/location reveal
- Event day mode
- Live screen moderation pipeline

Phase 6 - Memory + Intelligence

- Event Memory

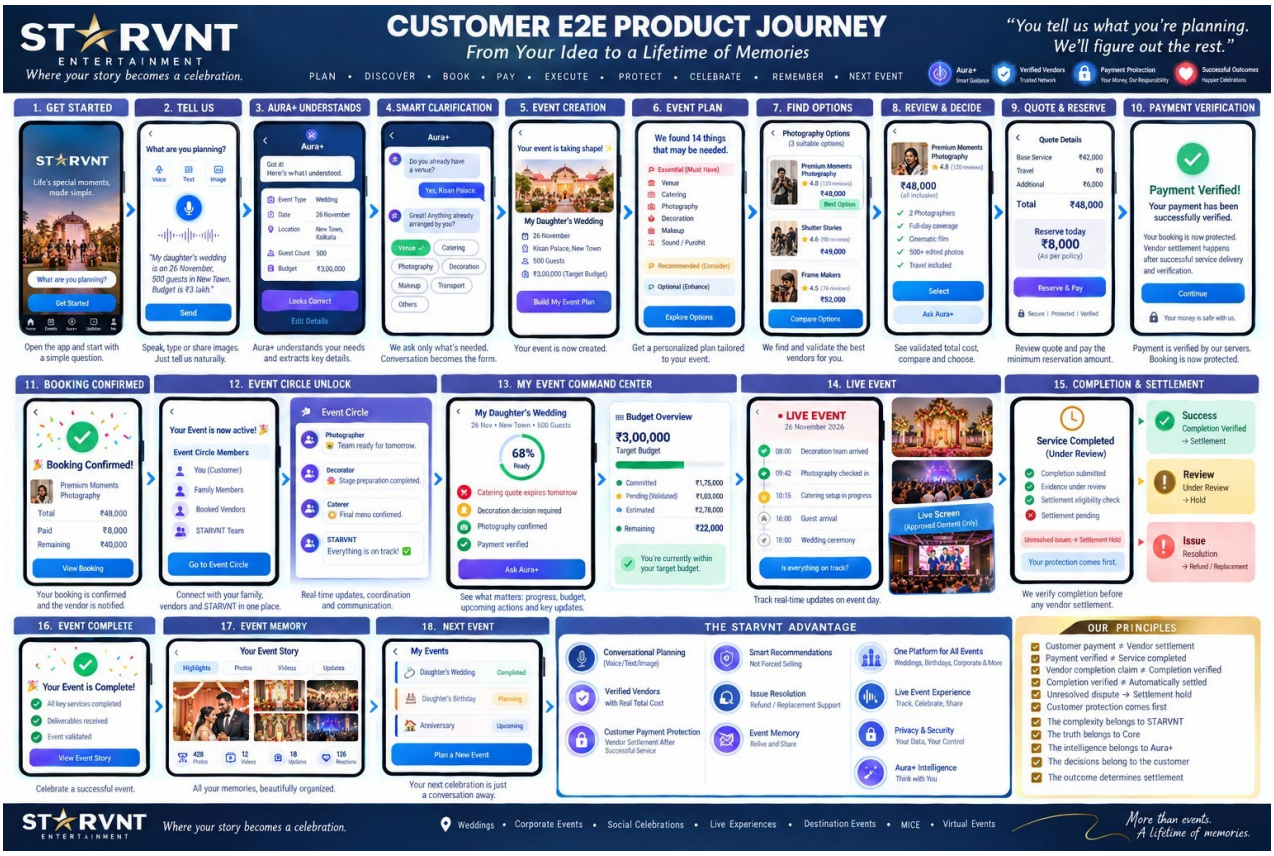
- Story assembly
- Cross-event customer context
- Aura+ “what am I missing?”
- On-track summaries
- Recovery recommendations

Definition of Done for each phase

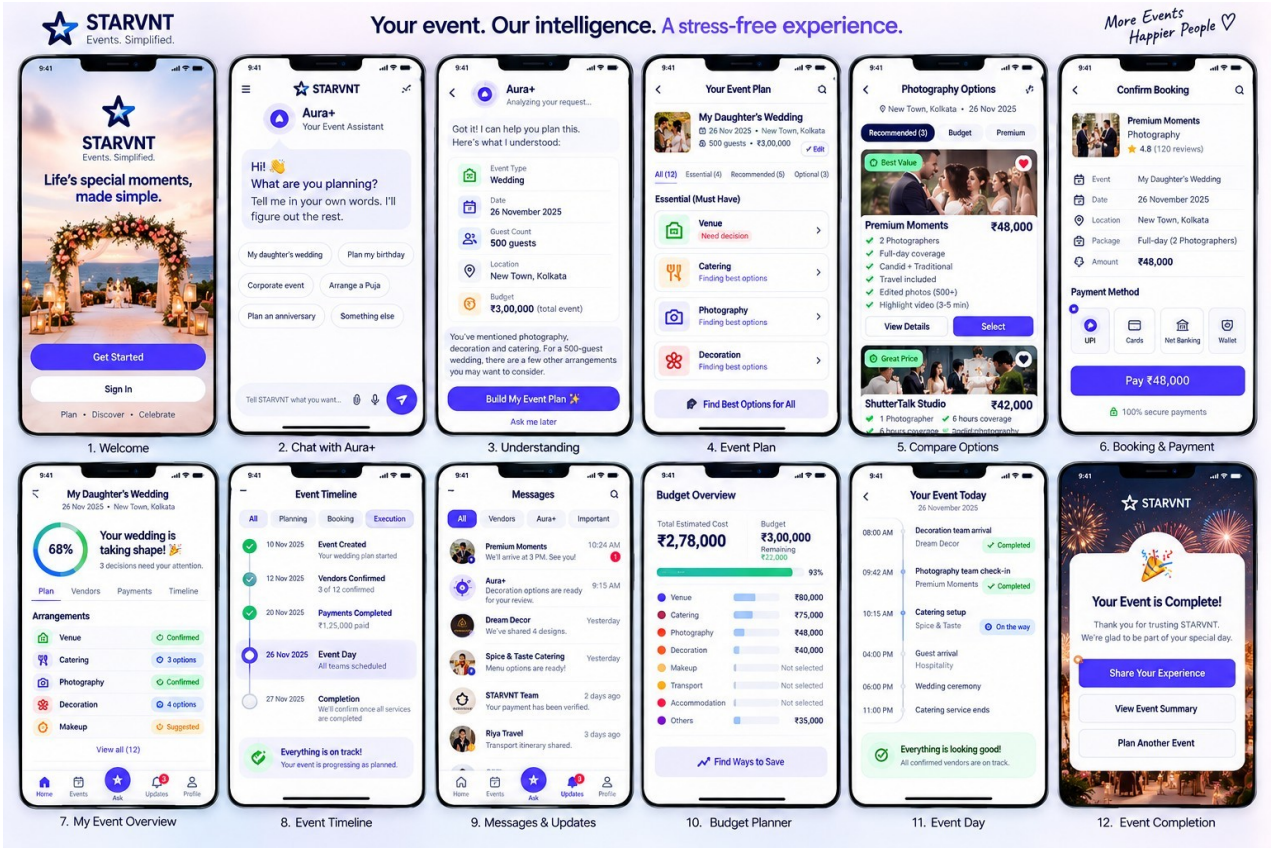
- Happy path works
- Failure path works
- Unauthorized path is blocked
- Retry path is idempotent
- Audit trail exists
- UI reconciles to server truth
- No critical state is client-authoritative
- Relevant E2E tests are automated

18. REFERENCE SCREENS AND PRODUCT LANGUAGE

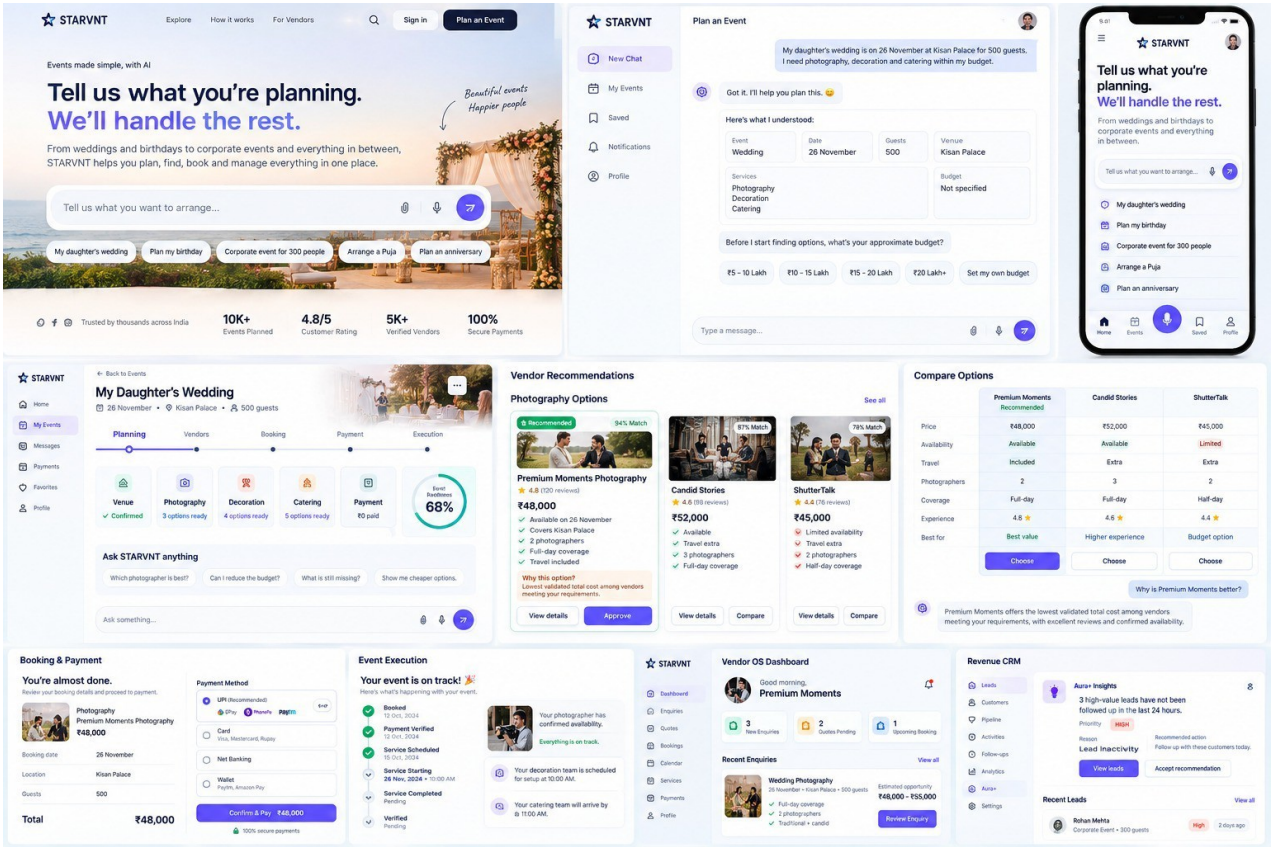
The following supplied images are product/design references. Developers should use them to understand intended information hierarchy, not as permission to hardcode state or data. Actual implementation must bind UI to authoritative API state and the state machines defined above.



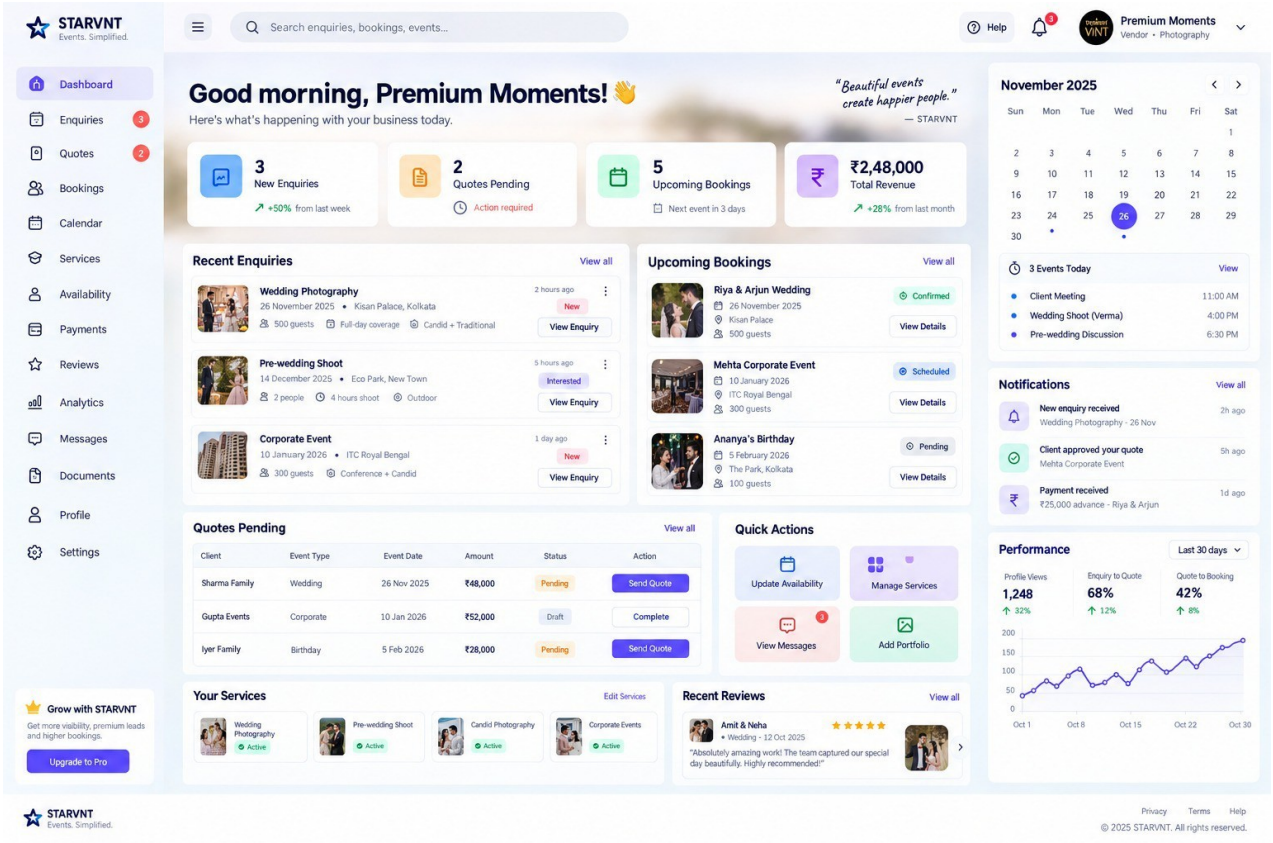
Reference screen set 1.



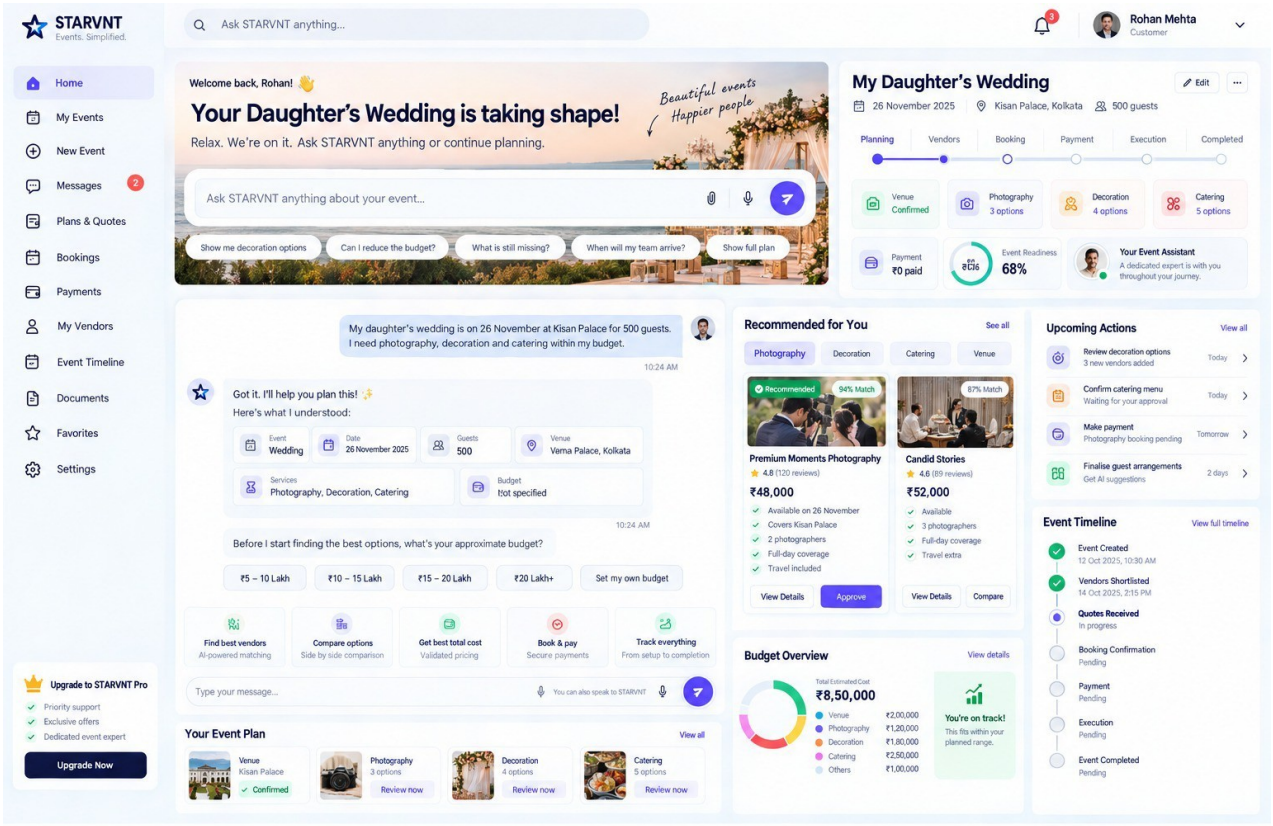
Reference screen set 2.



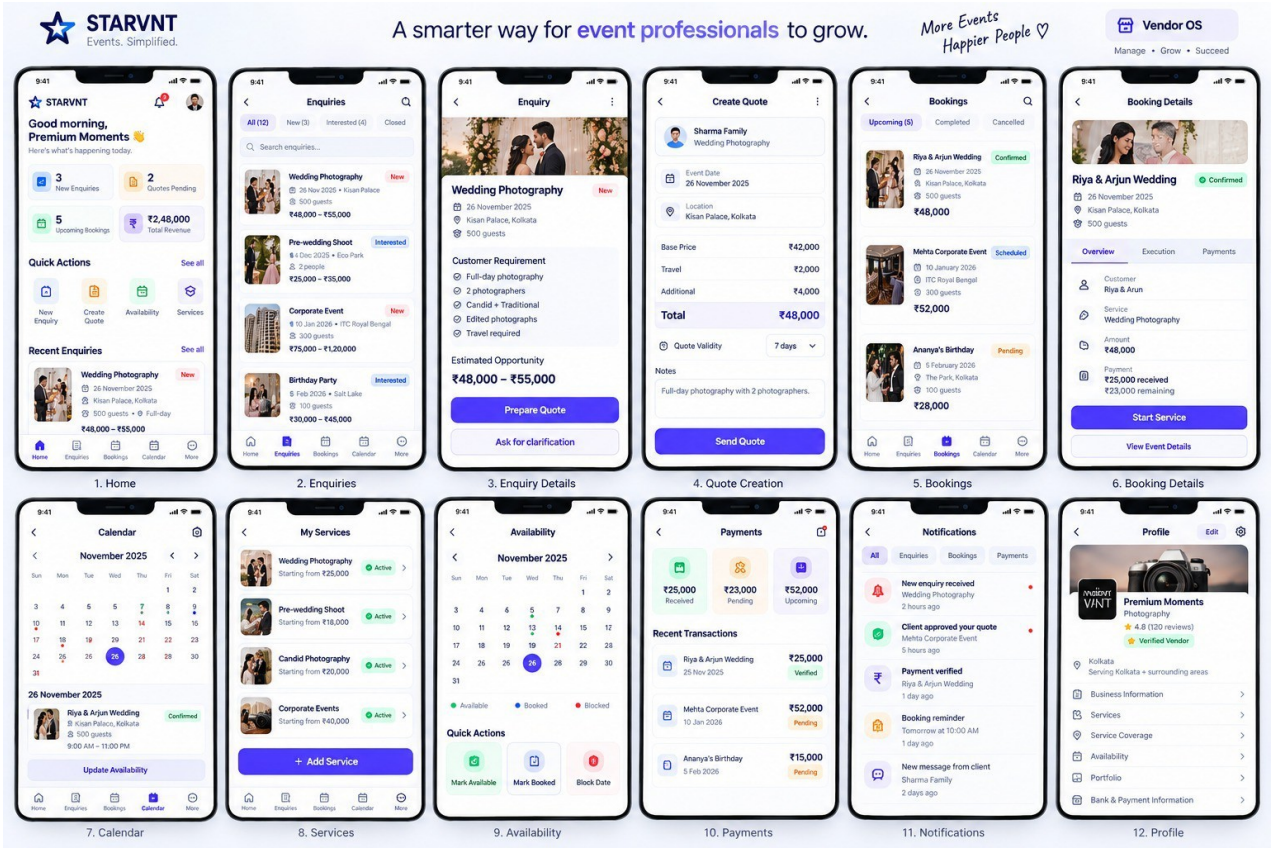
Reference screen set 3.



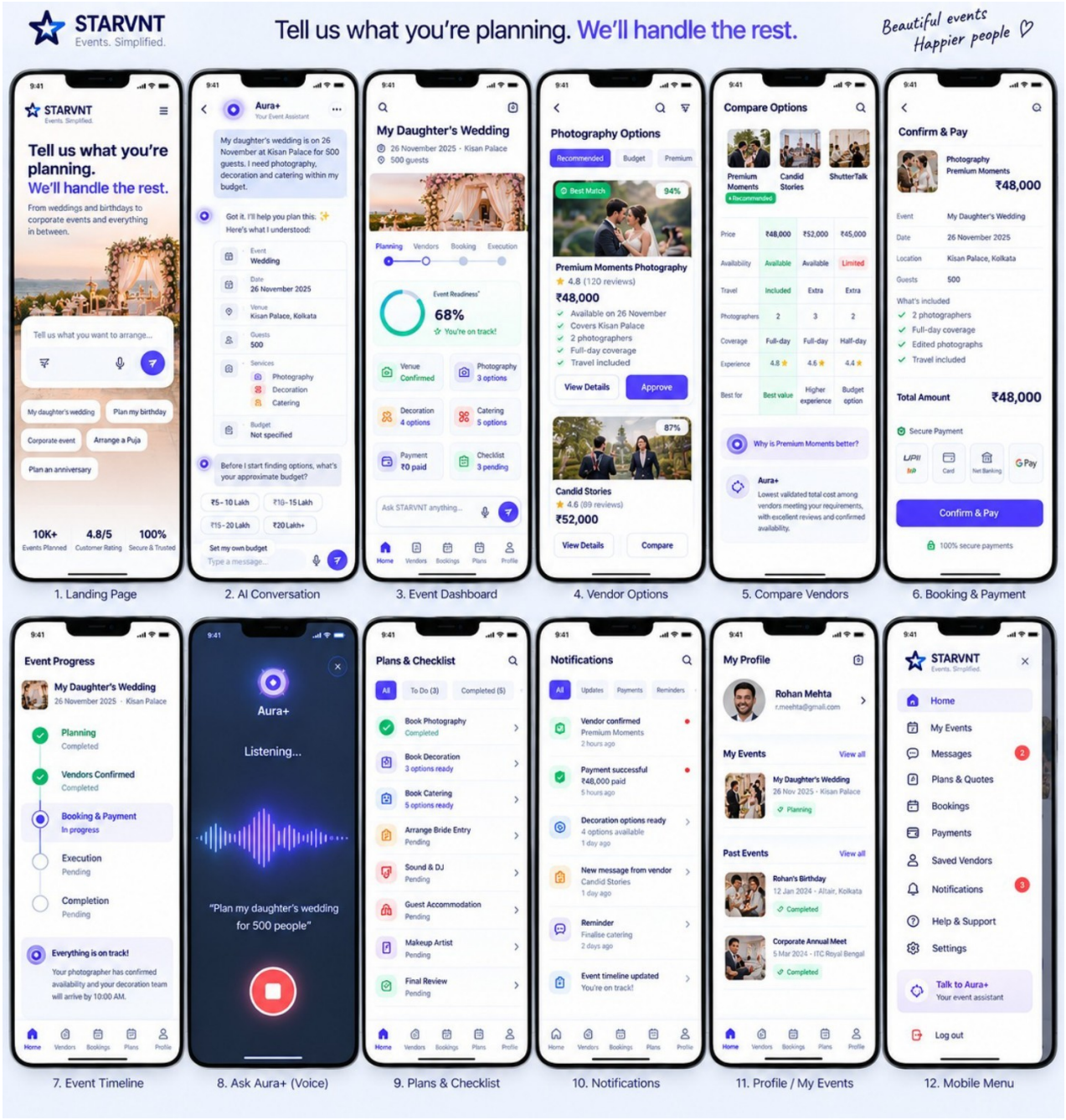
Reference screen set 4.



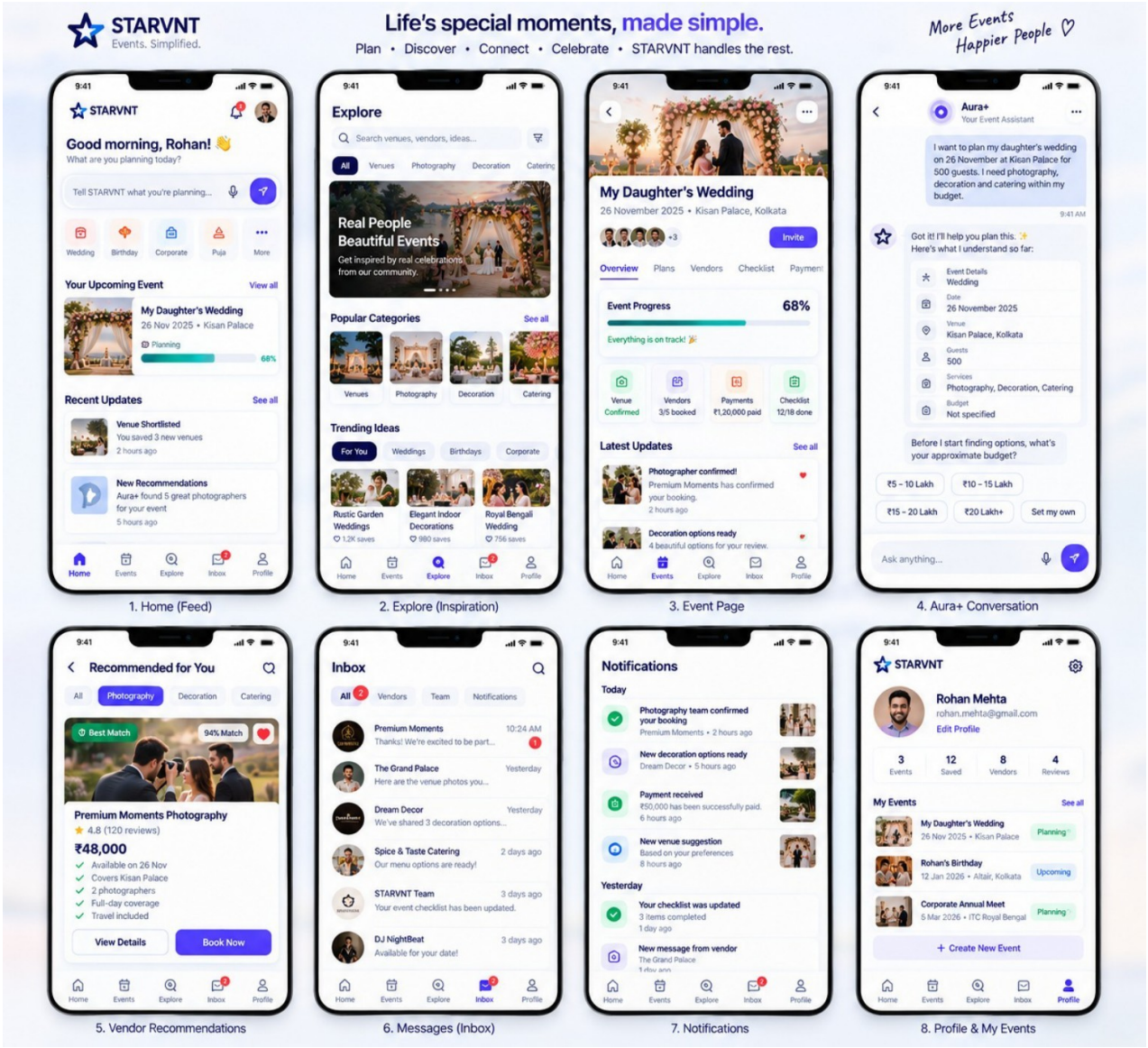
Reference screen set 5.



Reference screen set 6.



Reference screen set 7.



Reference screen set 8.

FINAL STARVNT ENGINEERING CONTRACT

1. CUSTOMER

The customer owns the decision.

2. AURA+

Aura+ understands, explains and recommends. It does not create authoritative business truth.

3. CORE

Core owns authoritative event, requirement, booking, payment, completion and settlement state.

4. POLICY

Policy determines which commercial and protection rules apply.

5. MONEY

Payment verification, custody and settlement execution are distinct concerns.

6. OUTCOME

Settlement follows validated service outcome under the configured policy and infrastructure.

7. PROTECTION

Unresolved applicable issues receive review before settlement proceeds.

8. UX

The customer experiences simplicity; STARVNT absorbs the operational complexity.

Final customer promise: “You tell us what you're planning. We'll figure out the rest.”